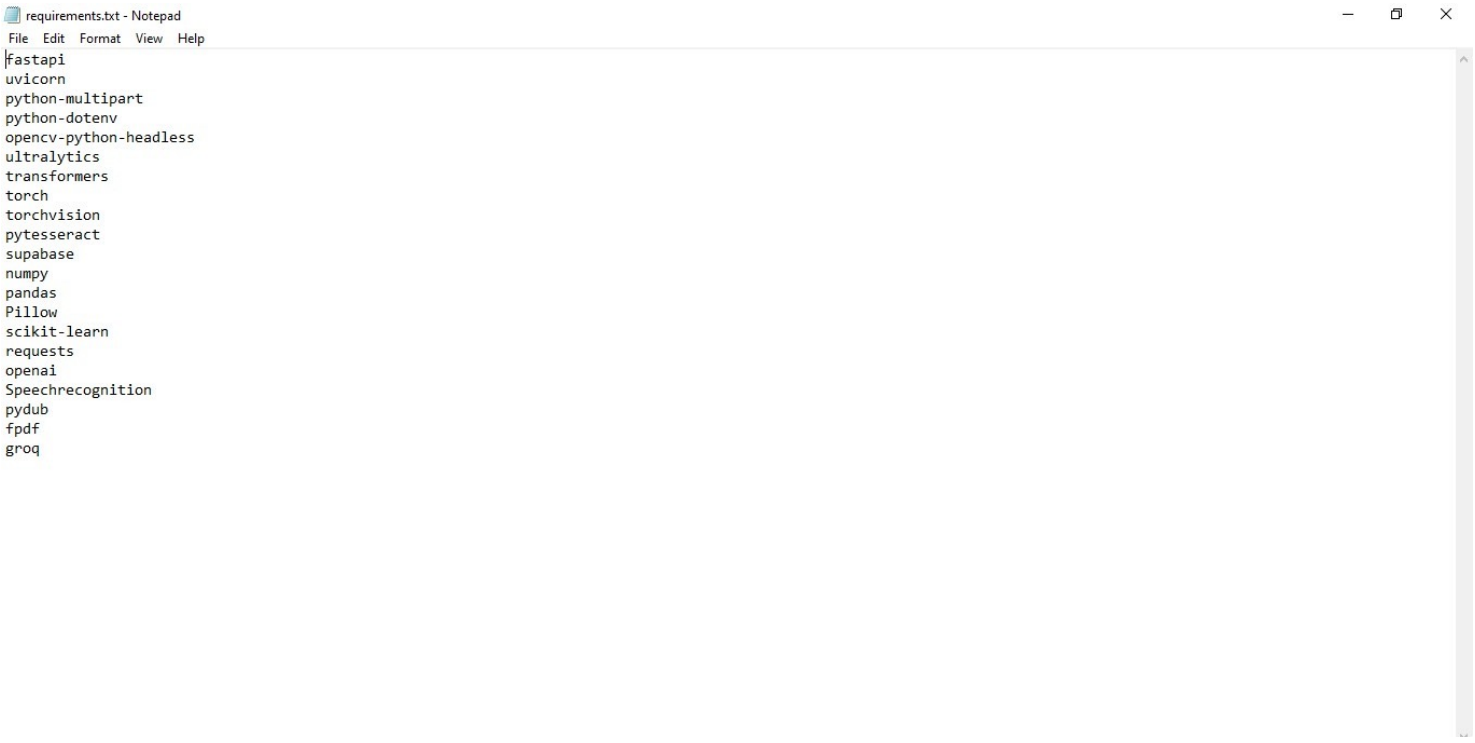
FOODVERSE AI

Multimodal Food Intelligence Agent

PROJECT DEVELOPMENT PHASE

Activity 1: Requirements Specification and Environment Setup



Activity 2: Food Recognition and Nutrition Module Development

main.py - Notepad

File Edit Format View Help

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routers.food_recognition import router as food_recognition_router
from backend.routers.food_preprocessing import router as food_preprocessing_router
from backend.routers.hygiene_monitor import router as hygiene_monitor_router
from backend.routers.kitchen_assistant import router as kitchen_assistant_router
from backend.routers.meal_planner import router as meal_planner_router


app = FastAPI(title="Foodverse AI")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(food_recognition_router)
app.include_router(food_preprocessing_router)
app.include_router(hygiene_monitor_router)
app.include_router(kitchen_assistant_router,
    prefix="/api/kitchen-assistant",
    tags=["Kitchen Assistant"]
)
app.include_router(
    meal_planner_router,
    prefix="/api/meal-planner",
    tags=["Personalized Meal Planner"]
)

@app.get("/")
def home():
    return {
        "message": "Welcome to Foodverse AI!",
        "status": "Backend is running"
    }
```

food_recognition.py - Notepad

File Edit Format View Help

```
from fastapi import APIRouter, UploadFile, File, HTTPException, Form
from pydantic import BaseModel
from typing import List, Optional
import os
import shutil
import base64
import json
import speech_recognition as sr
from pydub import AudioSegment
from PIL import Image
import io
import re
from groq import Groq
from dotenv import load_dotenv

from pathlib import Path

load_dotenv(
    Path(__file__).resolve().parents[1] / ".env",
    override=True
)

router = APIRouter(
    prefix="/api/food-recognition",
    tags=["Food Recognition"]
)

# --- CONFIG ---
# Initialize Groq Client
api_key = os.getenv("GROQ_API_KEY")
client = Groq(api_key=api_key) if api_key else None

MODEL_NAME = "qwen/qwen3.8-27b"

class NutritionInfo(BaseModel):
    calories: int
```

food_recognition.py - Notepad

File Edit Format View Help

```
class NutritionInfo(BaseModel):
    calories: int
    protein: float
    carbs: float
    fats: float
    fiber: float

@router.get("/test")
def test_food_recognition():
    return {
        "message": "Food Recognition API is working!",
        "status": "success"
    }
@router.post("/upload")
async def upload_food_image(file: UploadFile = File(...)):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(
            status_code=400,
            detail="Please upload an image file"
        )

    image_bytes = await file.read()

    if not image_bytes:
        raise HTTPException(
            status_code=400,
            detail="Uploaded image is empty"
        )

    if client is None:
        raise HTTPException(
            status_code=500,
            detail="GROQ_API_KEY is missing. Configure it in your .env file."
        )
```

food_recognition.py - Notepad

File Edit Format View Help

```
    try:
        encoded_image = base64.b64encode(image_bytes).decode("utf-8")

        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "text",
                            "text": (
                                "Identify the food in this image. "
                                "Give the likely food name, visible "
                                "ingredients if identifiable, and a "
                                "brief explanation. If uncertain, say so."
                            )
                        },
                        {
                            "type": "image_url",
                            "image_url": {
                                "url": (
                                    f"data:{file.content_type}; base64,"
                                    f"{encoded_image}"
                                )
                            }
                        }
                    ]
                }
            ],
            temperature=0.2,
            max_tokens=400
        )

        recognition = response.choices[0].message.content
```

food_recognition.py - Notepad

File Edit Format View Help

```
        recognition = response.choices[0].message.content

        return {
            "message": "Image analyzed successfully",
            "filename": file.filename,
            "food_recognition": recognition,
            "status": "success"
        }

    except Exception as e:
        raise HTTPException(
            status_code=502,
            detail=f"Food recognition failed: {str(e)}"
        )
@router.post("/nutrition")
async def estimate_nutrition(food_name: str):
    if not client:
        raise HTTPException(
            status_code=500,
            detail="GROQ_API_KEY is missing"
        )

    try:
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {
                    "role": "user",
                    "content": (
                        f"Estimate nutrition for 100g of {food_name}. "
                        "Return only valid JSON with these fields: "
                        "dish_name, calories, protein, carbs, fats, "
                        "fiber, sugar, health_score, health_label, "
                        "reasoning, alternatives. "
                        "Use numeric values for nutrition and score. "
```

food_recognition.py - Notepad

File Edit Format View Help

```
    try:
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {
                    "role": "user",
                    "content": (
                        f"Estimate nutrition for 100g of {food_name}. "
                        "Return only valid JSON with these fields: "
                        "dish_name, calories, protein, carbs, fats, "
                        "fiber, sugar, health_score, health_label, "
                        "reasoning, alternatives. "
                        "Use numeric values for nutrition and score. "
                        "health_score must be 0-100. "
                        "health_label must be Healthy, Moderate, or Unhealthy."
                    )
                }
            ],
            temperature=0.2,
            max_tokens=400
        )

        result = response.choices[0].message.content
        return {
            "status": "success",
            "nutrition": json.loads(result)
        }

    except Exception as e:
        raise HTTPException(
            status_code=502,
            detail=f"Nutrition estimation failed: {str(e)}"
        )
```

Activity 3: Food Freshness and Quality Detection

food_preprocessing.py - Notepad

File Edit Format View Help

```
        # Reduce noise
        denoised = resized.filter(ImageFilter.MedianFilter(size=3))

        # Extract color features
        stats = ImageStat.Stat(denoised)
        mean_rgb = [round(value, 2) for value in stats.mean]
        stddev_rgb = [round(value, 2) for value in stats.stddev]

        # Estimate brightness and contrast
        grayscale = denoised.convert("L")
        gray_stats = ImageStat.Stat(grayscale)
        brightness = round(gray_stats.mean[0], 2)
        contrast = round(gray_stats.stddev[0], 2)

        # Estimate texture using edge strength
        edges = grayscale.filter(ImageFilter.FIND_EDGES)
        edge_strength = round(ImageStat.Stat(edges).mean[0], 2)

        return {
            "status": "success",
            "filename": file.filename,
            "original_size": {
                "width": original_width,
                "height": original_height
            },
            "processed_size": {
                "width": 224,
                "height": 224
            },
            "preprocessing": [
                "RGB color conversion",
                "Image resizing",
                "Median-filter noise reduction"
            ],
            "features": {
```

food_preprocessing.py - Notepad

File Edit Format View Help

```
from fastapi import APIRouter, UploadFile, File, HTTPException
from PIL import Image, ImageFilter, ImageStat
import io
from pydantic import BaseModel, Field

router = APIRouter(
    prefix="/api/food-preprocessing",
    tags=["Food Preprocessing"]
)


@router.post("/analyze")
async def preprocess_food_image(file: UploadFile = File(...)):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(
            status_code=400,
            detail="Please upload a valid image."
        )

    try:
        image_bytes = await file.read()

        if not image_bytes:
            raise HTTPException(
                status_code=400,
                detail="The uploaded image is empty."
            )

        # Load and standardize the image
        image = Image.open(io.BytesIO(image_bytes))
        image.load()
        original_width, original_height = image.size
        image = image.convert("RGB")

        # Resize to a standard size
        resized = image.resize((224, 224))
```

food_preprocessing.py - Notepad

File Edit Format View Help

```
                "Median-filter noise reduction"
            ],
            "features": {
                "mean_rgb": mean_rgb,
                "color_variation_rgb": stddev_rgb,
                "brightness": brightness,
                "contrast": contrast,
                "edge_strength": edge_strength
            },
            "note": (
                "These are basic visual features. "
                "They do not independently prove food freshness "
                "or safety."
            )
        }

    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=400,
            detail=f"Image preprocessing failed: {str(e)}"
        )
from collections import Counter


def classify_freshness(food_name: str, visual_signs: list = None):
    """
    Activity 3.2: Food Freshness Classification
    and Safety Recommendation Logic.
    """

    visual_signs = visual_signs or []
    signs = [str(sign).lower() for sign in visual_signs]

    spoiled_signs = [
```

food_preprocessing.py - Notepad

File Edit Format View Help

```
    ]

    moderate_signs = [
        "brown spots", "wrinkled", "slightly soft",
        "yellowing", "minor discoloration"
    ]

    if any(sign in item for sign in signs for item in spoiled_signs):
        freshness = "Spoiled"
        safety_status = "Unsafe"
        confidence = 90
        recommendation = (
            "Do not consume if spoilage is suspected. "
            "Discard the food safely."
        )

    elif any(sign in item for sign in signs for item in moderate_signs):
        freshness = "Moderate"
        safety_status = "Consume Soon"
        confidence = 75
        recommendation = (
            "Inspect carefully and store appropriately. "
            "Visual appearance alone cannot establish safety."
        )

    else:
        freshness = "Fresh"
        safety_status = "Safe to Eat"
        confidence = 60
        recommendation = (
            "No listed spoilage signs were provided. "
            "Check smell, texture, storage history and expiry. "
            "Appearance alone cannot confirm food safety."
        )
```

food_preprocessing.py - Notepad

File Edit Format View Help

```
        )

    return {
        "item_name": food_name,
        "freshness_level": freshness,
        "confidence": confidence,
        "safety_status": safety_status,
        "detected_signs": visual_signs,
        "reasoning": (
            "Classification based on the visual signs supplied; "
            "this is a preliminary estimate."
        ),
        "recommendation": recommendation
    }
class FreshnessRequest(BaseModel):
    food_name: str = Field(..., min_length=1)
    visual_signs: list[str] = Field(default_factory=list)


@router.post("/freshness")
async def check_food_freshness(request: FreshnessRequest):
    try:
        result = classify_freshness(
            food_name=request.food_name,
            visual_signs=request.visual_signs
        )
        return {
            "status": "success",
            "freshness_analysis": result
        }
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Freshness classification failed: {str(e)}"
        )
```

Activity 4: Restaurant Hygiene and Smart Kitchen

hygiene_monitor.py - Notepad

File Edit Format View Help

```
import base64
import json
import re
from typing import Optional

from fastapi import APIRouter, File, Form, HTTPException, UploadFile
from PIL import Image, ImageStat
import io

router = APIRouter(
    prefix="/api/hygiene",
    tags=["Restaurant Hygiene"]
)


def analyze_hygiene_image(image_bytes: bytes, context_text: str = ""):
    """Analyze basic image features and return a preliminary hygiene report."""

    try:
        image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
        image.load()

        width, height = image.size
        stats = ImageStat.Stat(image)
        brightness = round(sum(stats.mean) / 3, 2)

        # Basic rule-based baseline; this does not detect PPE or violations.
        return {
            "status": "success",
            "image_size": {"width": width, "height": height},
            "brightness": brightness,
            "context": context_text,
            "overall_score": 100,
            "floor_score": None,
            "violations": [],
            "ai summary": (
```

hygiene_monitor.py - Notepad

File Edit Format View Help

```
            "violations": [],
            "ai_summary": (
                "Image loaded successfully. Automated detection of gloves, "
                "hairnets, phone use, and surface cleanliness is not enabled. "
                "A human inspection is required before assigning a real "
                "hygiene compliance score."
            ),
            "analysis_type": "image validation baseline"
        }

    except Exception as e:
        raise HTTPException(
            status_code=400,
            detail=f"Unable to process image: {str(e)}"
        )


@router.post("/analyze")
async def analyze_hygiene(
    file: UploadFile = File(...),
    context_text: Optional[str] = Form(default="")
):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(
            status_code=400,
            detail="Please upload a valid image."
        )

    image_bytes = await file.read()

    if not image_bytes:
        raise HTTPException(status_code=400, detail="Uploaded image is empty.")

    return analyze_hygiene_image(image_bytes, context_text or "")
```

hygiene_monitor.py - Notepad

File Edit Format View Help

```
@router.post("/score")
async def calculate_hygiene_score(
    violations: str = Form(default="")
):
    import json
    from fastapi import HTTPException

    raw = violations.strip()

    if not raw:
        items = []
    else:
        try:
            parsed = json.loads(raw)
            if isinstance(parsed, list):
                items = parsed
            else:
                items = [raw]
        except json.JSONDecodeError:
            # Swagger may send plain text instead of JSON.
            items = [item.strip() for item in raw.split(",") if item.strip()]

    if not all(isinstance(item, str) for item in items):
        raise HTTPException(
            status_code=400,
            detail="Each violation must be text."
        )

    weights = {
        "no gloves": 20,
        "hair net missing": 20,
        "dirty floor": 15,
        "phone use": 15,
        "unsafe food handling": 30
    }
```

hygiene_monitor.py - Notepad

File Edit Format View Help

```
            items = [item.strip() for item in raw.split(",") if item.strip()]

    if not all(isinstance(item, str) for item in items):
        raise HTTPException(
            status_code=400,
            detail="Each violation must be text."
        )

    weights = {
        "no gloves": 20,
        "hair net missing": 20,
        "dirty floor": 15,
        "phone use": 15,
        "unsafe food handling": 30
    }

    report = []
    deduction = 0

    for item in items:
        name = item.strip()
        weight = weights.get(name.lower(), 10)
        deduction += weight
        report.append({
            "violation": name,
            "severity": "High" if weight >= 20 else "Medium",
            "score_deduction": weight
        })

    return {
        "status": "success",
        "overall_score": max(0, 100 - deduction),
        "violations": report,
        "ai_summary": "Score calculated from supplied violations; automatic visual detection is not performed."
    }
```

kitchen_assistant.py - Notepad

File Edit Format View Help

```
import os
import json
import base64
from typing import Optional

from fastapi import APIRouter, Form, File, UploadFile, HTTPException
from pydantic import BaseModel
from groq import Groq

router = APIRouter()

MODEL_NAME = os.getenv("GROQ_MODEL", "qwen/qwen3.8-27b")
client = Groq(api_key=os.getenv("GROQ_API_KEY"))


class RecipeGenerationResponse(BaseModel):
    dish_name: str
    ingredients: list[str]
    steps: list[str]


@router.post("/generate-recipe", response_model=RecipeGenerationResponse)
async def generate_recipe(
    text_input: Optional[str] = Form(None),
    file: Optional[UploadFile] = File(None),
):
    temp_path = None

    try:
        if not text_input and not file:
            return RecipeGenerationResponse(
                dish_name="No Input",
                ingredients=[],
                steps=["Please provide ingredients or upload a food image."]
            )
```

kitchen_assistant.py - Notepad

File Edit Format View Help

```
        messages = [
            {
                "role": "system",
                "content": (
                    "You are an AI chef. Generate a recipe using the user's "
                    "ingredients or food image. Return valid JSON with exactly "
                    "these fields: dish_name, ingredients (list of strings), "
                    "and steps (list of strings). Do not invent certainty "
                    "about ingredients that cannot be identified."
                )
            }
        ]

        if file:
            if not file.content_type or not file.content_type.startswith("image/"):
                raise HTTPException(
                    status_code=400,
                    detail="Please upload a valid image file."
                )

            image_bytes = await file.read()
            if not image_bytes:
                raise HTTPException(
                    status_code=400,
                    detail="The uploaded image is empty."
                )

            b64 = base64.b64encode(image_bytes).decode("utf-8")
            image_url = (
                f"data:{file.content_type}; base64,{b64}"
            )

            user_content = [
                {
                    "type": "text",
```

kitchen_assistant.py - Notepad

File Edit Format View Help

```
        else:
            messages.append({
                "role": "user",
                "content": (
                    f"Generate a detailed recipe based on these ingredients "
                    f"or requirements: {text_input}"
                )
            })

        completion = client.chat.completions.create(
            model=MODEL_NAME,
            messages=messages,
            response_format={"type": "json_object"},
            max_tokens=1200,
        )

        result = json.loads(
            completion.choices[0].message.content
        )

        return RecipeGenerationResponse(
            dish_name=result.get("dish_name", "Custom Recipe"),
            ingredients=result.get("ingredients", []),
            steps=result.get("steps", [])
        )

    except HTTPException:
        raise

    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=f"Recipe generation failed: {str(exc)}"
        )
```

Activity 5: Personalized Meal Plan Development



meal_planner.py - Notepad

File Edit Format View Help

```
class MealAlternatives(BaseModel):
    original_dish: str
    alternatives: List[MealDetails]


class DailyMealPlan(BaseModel):
    breakfast: MealDetails
    mid_morning_snack: MealDetails
    lunch: MealDetails
    evening_snack: MealDetails
    dinner: MealDetails
    daily_totals: Dict[str, str]
    hydration_target: str


class WeeklyReport(BaseModel):
    adherence_score: int
    calorie_balance: str
    macro_balance: str
    consistency: str
    concern_areas: List[str]
    highlights: List[str]
    next_week_suggestions: List[str]
    motivation: str


class FullMealPlanResponse(BaseModel):
    user_profile_summary: str
    daily_plan: DailyMealPlan


@router.post("/profile", tags=["Personalized Meal Planner"])
async def process_user_profile(profile: UserProfile):
    return {
        "message": "User profile validated successfully",
```

meal_planner.py - Notepad

File Edit Format View Help

```
@router.post("/generate-daily-plan", response_model=FullMealPlanResponse,
             tags=["Personalized Meal Planner"])
async def generate_daily_plan(profile: UserProfile):
    prompt = f"""
Create a one-day meal plan for this user:
{profile.model_dump_json()}

Respect all listed allergies and dietary preferences.
Return only valid JSON matching this structure:
{{
  "user_profile_summary": "brief summary",
  "daily_plan": {{
    "breakfast": {{"dish_name":"", "description":"", "ingredients":[],
      "calories":0, "macros":{{"protein":"", "carbs":"", "fats":""}},
      "reasoning":"", "portion_size":"", "prep_style":""}},
    "mid_morning_snack": {{"dish_name":"", "description":"", "ingredients":[],
      "calories":0, "macros":{{"protein":"", "carbs":"", "fats":""}},
      "reasoning":"", "portion_size":"", "prep_style":""}},
    "lunch": {{"dish_name":"", "description":"", "ingredients":[],
      "calories":0, "macros":{{"protein":"", "carbs":"", "fats":""}},
      "reasoning":"", "portion_size":"", "prep_style":""}},
    "evening_snack": {{"dish_name":"", "description":"", "ingredients":[],
      "calories":0, "macros":{{"protein":"", "carbs":"", "fats":""}},
      "reasoning":"", "portion_size":"", "prep_style":""}},
    "dinner": {{"dish_name":"", "description":"", "ingredients":[],
      "calories":0, "macros":{{"protein":"", "carbs":"", "fats":""}},
      "reasoning":"", "portion_size":"", "prep_style":""}},
    "daily_totals": {{"calories":"", "protein":"", "carbs":"", "fats":""}},
    "hydration_target":""
  }}
}}
Treat calorie and nutrition estimates as approximate.
Do not claim to provide medical advice.
"""
```

meal_planner.py - Notepad

File Edit Format View Help

```
    try:
        completion = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {"role": "system", "content": "You are a careful meal-planning assistant."},
                {"role": "user", "content": prompt},
            ],
            response_format={"type": "json_object"},
            max_tokens=3000,
        )
        data = json.loads(completion.choices[0].message.content)
        return FullMealPlanResponse(**data)

    except Exception as exc:
        raise HTTPException(
            status_code=502,
            detail=f"Meal plan generation failed: {str(exc)}",
        )


@router.post("/alternatives", response_model=MealAlternatives,
             tags=["Personalized Meal Planner"])
async def suggest_alternatives(
    dish_name: str,
    profile: UserProfile,
):
    prompt = f"""
Suggest 3 alternatives to this dish: {dish_name}.
User profile: {profile.model_dump_json()}
Respect allergies and dietary preferences.
Return JSON with original_dish and alternatives.
Each alternative must have dish_name, description, ingredients,
calories, macros (protein, carbs, fats), reasoning, portion_size,
and prep_style. Use approximate nutrition estimates.
"""
    try:
```

meal_planner.py - Notepad

File Edit Format View Help

```
and prep_style. Use approximate nutrition estimates.
"""
    try:
        completion = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {"role": "system", "content": "You are a careful meal-planning assistant."},
                {"role": "user", "content": prompt},
            ],
            response_format={"type": "json_object"},
            max_tokens=2000,
        )
        data = json.loads(completion.choices[0].message.content)
        return MealAlternatives(**data)

    except Exception as exc:
        raise HTTPException(
            status_code=502,
            detail=f"Alternative generation failed: {str(exc)}",
        )
@router.post(
    "/generate",
    response_model=FullMealPlanResponse,
    tags=["Personalized Meal Planner"]
)
async def generate_comprehensive_plan(
    age: int = Form(...),
    gender: str = Form(...),
    height: float = Form(...),
    weight: float = Form(...),
    activity_level: str = Form("Moderate"),
    medical_conditions: str = Form("None"),
    goals: str = Form(...),
    diet_type: str = Form("Standard"),
    cuisine: str = Form("Global"),
    allergies: str = Form("None"),
```

meal_planner.py - Notepad

File Edit Format View Help

```
):
    prompt = f"""
You are a professional AI dietitian. Generate a personalized
one-day meal plan using the user's information.

USER PROFILE:
Age: {age}
Gender: {gender}
Height: {height} cm
Weight: {weight} kg
Activity level: {activity_level}
Medical conditions: {medical_conditions}
Goals: {goals}
Diet type: {diet_type}
Cuisine: {cuisine}
Allergies: {allergies}
Recent history: {recent_history}
Recent progress: {weight_change}

Respect all allergies and dietary restrictions.
Provide breakfast, mid-morning snack, lunch, evening snack,
and dinner. Include ingredients, approximate calories,
macros, reasoning, portion size, and preparation style.
Include daily nutrition totals and a hydration target.
Treat nutrition values as estimates, not medical advice.

Return only valid JSON with this structure:
{{
  "user_profile_summary": "summary",
  "daily_plan": {{
    "breakfast": {{
      "dish_name": "name",
      "description": "description",
      "ingredients": ["ingredient"],
      "calories": 300,
      "macros": {{
```

meal_planner.py - Notepad

File Edit Format View Help

```
    }},
    "reasoning": "reason",
    "portion_size": "one serving",
    "prep_style": "cooked"
  }},
  "mid_morning_snack": {{
    "dish_name": "name",
    "description": "description",
    "ingredients": ["ingredient"],
    "calories": 150,
    "macros": {{
      "protein": "5g",
      "carbs": "20g",
      "fats": "5g"
    }},
    "reasoning": "reason",
    "portion_size": "one serving",
    "prep_style": "fresh"
  }},
  "lunch": {{
    "dish_name": "name",
    "description": "description",
    "ingredients": ["ingredient"],
    "calories": 450,
    "macros": {{
      "protein": "20g",
      "carbs": "55g",
      "fats": "15g"
    }},
    "reasoning": "reason",
    "portion_size": "one serving",
    "prep_style": "cooked"
  }},
  "evening_snack": {{
    "dish_name": "name",
    "description": "description",
```

meal_planner.py - Notepad

File Edit Format View Help

```
    "daily_totals": {{
      "calories": "1500 kcal (estimated)",
      "protein": "65g",
      "carbs": "175g",
      "fats": "50g"
    }},
    "hydration_target": "Individualized daily water goal"
  }}
}}
"""
    try:
        completion = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {
                    "role": "system",
                    "content": (
                        "You generate structured, personalized meal plans. "
                        "Respect allergies and dietary restrictions."
                    ),
                },
                {"role": "user", "content": prompt},
            ],
            response_format={"type": "json_object"},
            max_tokens=3000,
        )

        data = json.loads(completion.choices[0].message.content)
        return FullMealPlanResponse(**data)

    except Exception as exc:
        raise HTTPException(
            status_code=502,
            detail=f"Comprehensive meal plan generation failed: {str(exc)}",
        )
```